How to Run the Multi-Domain RAG Assistant on Your PC

Multi-Domain Agentic RAG Assistant · Group 2 · final version v2.9

This guide takes you from a fresh copy of the code to the assistant running in your own browser. It is written for Windows; the macOS / Linux commands are given where they differ. Allow about 20 minutes of setup, plus a one-time index build (see step 6).

---

1. What you need

Item	Details
Computer	Windows 10/11, macOS or Linux · 8 GB RAM (16 GB recommended) · no GPU needed
Free disk space	about 5 GB (Python packages ≈ 3 GB, downloaded data and model ≈ 1.6 GB, index ≈ 170 MB)
Python	**3.12** (tested with 3.12.10) from https://www.python.org/downloads/
Internet	needed for installing, the first start, and every question (the language model runs on Groq's servers)
Groq API key	your own free key (step 3). The project's key is **not** included and must never be shared.
Git	optional, only if you clone instead of downloading a ZIP

> **Windows:** when installing Python, tick **"Add python.exe to PATH"** on the first screen.

---

2. Get the code

Repository: https://github.com/suneel130588-art/IITH_MultiDomain_RAG_Agents_Project_Group_2 (if it is not visible to you, ask the team for access or for a ZIP).

Option A, with Git:

```
git clone https://github.com/suneel130588-art/IITH_MultiDomain_RAG_Agents_Project_Group_2.git
cd IITH_MultiDomain_RAG_Agents_Project_Group_2
```

Option B, ZIP: on the repository page click **Code** → **Download ZIP**, unzip it and open a terminal in the unzipped folder.

Keep the folder structure as it is. The web app loads the notebook from `../notebooks/`:

```
IITH_MultiDomain_RAG_Agents_Project_Group_2/
├── notebooks/MultiDomain_RAG_Agents_v2.9_final.ipynb   ← the agents' code (the app runs it)
├── web_app/
│   ├── web_app.py            ← web server
│   ├── MultiDomainAgent.py   ← loads the notebook's pipeline
│   ├── requirements.txt      ← Python packages
│   ├── .env.example          ← template for your API key
│   └── static/               ← web pages (login, assistant, test questions)
└── docs/                     ← report, presentation, data
```

---

3. Get a Groq API key (free)

1. Go to https://console.groq.com and sign in (Google or e-mail).
2. Open **API Keys** → **Create API Key**, give it a name, and copy the key. It starts with `gsk_`.
3. Keep it private. You will paste it into a file in step 5.

---

4. Create a Python environment and install the packages

Open **PowerShell** (Windows) or **Terminal** (macOS/Linux) and go into the `web_app` folder:

Windows (PowerShell):

```
cd web_app
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

macOS / Linux:

```
cd web_app
python3 -m venv .venv
.venv/bin/python -m pip install --upgrade pip
.venv/bin/python -m pip install -r requirements.txt
```

Installation takes 5–15 minutes; PyTorch (used by the embedding model) is the largest download.

---

5. Add your API key

Copy the template to a file named `.env` **inside the** `web_app` **folder**, then put your key in it.

Windows:

```
copy .env.example .env
notepad .env
```

macOS / Linux:

```
cp .env.example .env
nano .env
```

The file must contain one line with your own key:

```
GROQ_API_KEY=gsk_your_own_key_here
```

Save and close. `.env` is ignored by Git, so it is never uploaded.

---

6. (Recommended) Skip the long first build

On the first start the app builds its search index: it downloads the RAGBench data, cuts it into **14,704 chunks** and embeds each one. On a normal laptop CPU this takes **about 1 hour** (a few minutes with an NVIDIA GPU). It happens only once; the index is saved in `web_app/chroma_multidomain/`.

To skip it, ask the team for the ready-made **`chroma_multidomain`** folder (≈ 170 MB, not in Git) and copy it into `web_app/`, so the result is `web_app/chroma_multidomain/`.

---

7. Start the application

From the `web_app` folder:

Windows:

```
.\.venv\Scripts\python.exe web_app.py --host 127.0.0.1 --port 8001
```

macOS / Linux:

```
.venv/bin/python web_app.py --host 127.0.0.1 --port 8001
```

What you will see:

```
Groq key loaded: True
Embedding device: cpu
...
covidqa   (biomedical): 200 rows -> 713 chunks
cuad      (legal): 200 rows -> 8430 chunks
finqa     (finance): 200 rows -> 1043 chunks
techqa    (customer_support): 200 rows -> 3701 chunks
hotpotqa  (general): 200 rows -> 817 chunks
161 contracts registered for contract-scoped legal search.
Loaded 14704 chunks from 5 domains.
Embedding 14704 chunks into Chroma at ... (first run)...     ← first start only (about 1 hour on
CPU)
Loaded 14704 chunks from Chroma at ...                       ← every later start (about 30
seconds)
Multi-domain RAG chat available at http://127.0.0.1:8001
```

Leave this window open while you use the app. To stop it, press **Ctrl + C**.

> On a CPU the app also prints a warning that embedding takes about 1–1.5 hours. That is expected on the first start.

---

8. Use it in the browser

Open **http://127.0.0.1:8001**.

Page	What to do
Assistant	type a question, or click an example; watch the five agents work; read the answer with its `[RAG-…]` citations and the sources on the right
Test questions	the 30 benchmark questions (6 per domain); click **Ask** on any of them
Dataset (Excel)	download the indexed data (200 rows × 5 domains) to try more questions

Questions to try first:

- *What is the incubation period of MERS?* (biomedical)
- *In the MOELIS — STRATEGIC ALLIANCE AGREEMENT, is one party required to deposit its source code into escrow?* (legal: name the contract)
- *Is it safe to take warfarin together with aspirin?* (medicine check with OpenFDA; expert review flagged)

A typical answer takes 5–30 seconds.

---

9. Optional: add a login

Without a password the app opens directly. To require a login (recommended before sharing it with anyone), set a password before starting:

Windows (PowerShell):

```
$env:APP_USER = "iith"
$env:APP_PASSWORD = "choose-a-password"
.\.venv\Scripts\python.exe web_app.py --host 127.0.0.1 --port 8001
```

macOS / Linux:

```
APP_USER=iith APP_PASSWORD=choose-a-password .venv/bin/python web_app.py --host 127.0.0.1 --port 8001
```

To let people on other computers use it, put a tunnel such as ngrok in front of port 8001, **always with a login set**: every question uses your Groq key.

---

10. Troubleshooting

Problem	Fix
`python` is not recognized	Reinstall Python and tick "Add python.exe to PATH", or use `py -3.12` instead of `python`
`RuntimeError: GROQ_API_KEY is not set`	`.env` is missing or in the wrong folder. It must be `web_app/.env`, with the line `GROQ_API_KEY=gsk_...`
`Groq key loaded: False`	The key does not start with `gsk_`; copy it again from console.groq.com
The first start runs for a long time	Normal on CPU (about 1 hour, once). Use the ready-made `chroma_multidomain` folder (step 6)
`Address already in use` / port busy	Use another port, e.g. `--port 8002`, and open that address
An answer fails with "rate limit"	Groq's free tier has per-minute limits; wait a minute and ask again
Download errors from Hugging Face	Check the internet connection (or proxy) and start again; finished downloads are reused
The notebook file is not found	Keep the folder structure from step 2; `web_app` and `notebooks` must sit side by side
Errors installing packages	Use Python 3.12, upgrade pip (step 4) and install again

11. Alternative: run the notebook in Google Colab

No local install is needed:

1. Open `notebooks/MultiDomain_RAG_Agents_v2.9_final.ipynb` in https://colab.research.google.com (**File** → **Upload notebook**).
2. **Runtime** → **Change runtime type** → **T4 GPU** (the index then builds in 2–3 minutes).
3. Add your key under **Secrets** (key icon on the left) with the name `GROQ_API_KEY`, and allow notebook access.
4. **Runtime** → **Run all.** The last cell prints a public Gradio chat link.

Security reminders

- Never put a real API key in code, notebooks, screenshots or Git. Only `web_app/.env` (ignored by Git) holds it.
- This is a research prototype: answers come only from the RAGBench benchmark data and are not medical, legal, financial or technical advice.